

DocMind 项目面试题

1. 你的 Agentic RAG 和传统 RAG 有什么本质区别？为什么要做这个改进？

回答思路：先讲痛点，再讲方案，最后讲效果。

传统 RAG 的检索路径是**硬编码**的：所有问题进来，都走同一条 向量检索 → 拼 Prompt → 生成回答的固定流水线。这在实际场景中三个致命问题：

- 1. 检索策略不可变：用户问"第 12 条第 3 款怎么规定的"这种精确条款查询，走向量语义检索效果很差，应该走 BM25 关键词检索；但传统 RAG 没有选择权。
- 2. 无法多路召回：有些问题需要同时命中内部知识库和外部网页（比如"最新数据安全法有什么变化"），传统 RAG 只有单一检索路径。
- 3. 无法自我纠错：生成的答案可能和检索到的内容不一致，传统 RAG 生成就结束了，没有反思环节。

DocMind 的做法是用 **LLM Agent 替代硬编码路由**。具体实现上，我基于 Spring AI 的 ChatClient+Function Calling 机制，将 doc_search（向量检索）、keyword_search（BM25）、web_search（联网搜索）、recall_memory（长期记忆）注册为 LLM 可调用的工具。LLM 在运行时分析问题特征，自主决定调用哪些工具、调用几次、是否追加检索。

举个具体例子：

问题	LLM 决策	原因
"权限系统怎么配置？"	doc_search+ keyword_search	通用知识，语义 + 精确双路
"第 12 条第 3 款的规定"	keyword_search 优先	精确条款，关键词更准
"2026 年最新数据安全法"	三路全开 + web_search	时效性问题，需要联网
"继续说刚才那个限制"	recall_memory+ doc_search	追问，先召回上下文

同时我设计了 QueryRouter 规则路由作为**降级兜底**——如果 LLM Function Calling 失败（模型超时、返回格式异常），系统无缝切换到基于正则匹配 + LLM 意图分类的规则路由，保证检索链路零中断。

效果：不同类型的问题走不同的检索链路，精确类问题的召回率显著提升；同时降级机制保证了生产环境的稳定性。

2. 详细讲讲你的七步检索链路，每一步解决了什么问题？

回答思路：按流程逐步展开，每步讲清楚“为什么需要”和“怎么做的”。

整条链路是：**改写** → **工具选择** → **多路召回** → **RRF 融合** → **Cross-Encoder 精排** → **上下文压缩** → **自检纠错**。

Step 1 : Query Rewrite（查询改写）

问题：用户的提问往往是口语化的，比如“这个规范怎么配”，直接拿去做向量检索，和文档中“该规范的配置步骤与参数说明”的语义距离较大，召回率低。

方案：调用 LLM 将口语化问题改写为检索优化表达。我做了两个边界处理：

- 短查询（ ≤ 10 字）跳过改写，因为改写可能引入歧义
- 改写结果超过 200 字则截断，避免过度扩展
- LLM 调用失败时回退原始查询

Step 2 : LLM-Driven Tool Selection（动态工具选择）

上一题已经讲过，这里 LLM 通过 Function Calling 自主选择工具组合。

Step 3 : RRF 倒数秩融合

问题：向量检索返回的是 COSINE 相似度分数（0~1），BM25 返回的是 TF-IDF 分数（范围不固定），网页搜索返回的是 Tavily 的相关性分数。三路结果的**评分体系完全不同**，不能简单比较或加权。

方案：使用 RRF（Reciprocal Rank Fusion）算法：

$$\text{score}(d) = \sum 1 / (k + \text{rank_i}(d))$$

RRF 只依赖排名位置而非绝对分数，天然消除了异构评分体系的差异。k=60（可动态配置）控制头部偏好程度。同时我做了**跨路去重**——用内容前 50 字作为去重键，如果同一文档同时被向量和 BM25 检索到，标记为 HYBRID 源并合并分数，这种跨路命中的文档往往相关性最高。

Step 4 : Cross-Encoder 精排

问题：RRF 融合后仍有 20+ 候选文档，但 LLM 的 Context Window 有限，需要精选最相关的 Top-K。向量检索用的是 Bi-Encoder（query 和 doc 分别编码再算距离），精度不如 Cross-Encoder。

方案：调用 DashScope 的 gte-rerankCross-Encoder 模型，对每个 (query, chunk)对逐一计算细粒度相关性分数，取 Top-6。

降级设计：如果 rerank 模型不可用，降级为 60% 原始分 + 40% 关键词命中率的混合评分，不至于链路断裂。

Step 5 : 上下文压缩

问题：即使精排后只有 6 个 chunk，总字符数可能仍然很大，浪费 Token 预算。

方案：三步压缩——前缀去重（50 字匹配）→ 单片截断（800 字上限）→ 总量控制（不超过 maxTokens × 2 字符）。保留 rerankScore 最高的切片。

Step 6 : Prompt 组装 + 安全护栏

组装最终 Prompt 时注入：带来源标注的参考文档、用户画像（角色/领域/偏好）、长期记忆上下文、最近对话历史。

安全护栏做两件事：

4. 紧急关键词检测：匹配"火灾、中毒、自杀"等 15 个紧急类别，命中时附加安全提示
5. 置信度兜底：取 max(rerankScore)作为置信度，低于阈值（默认 0.3）触发兜底回答，明确告知用户"当前知识库中未找到高度相关的参考内容"，不编造答案

Step 7 : 流式输出 + Self-Reflection

SSE 字符级流式输出后，LLM 对自己的回答从**事实一致性、完整性、来源匹配、表达质量**四个维度打分（0~10 分）。置信度 < 0.7 触发重新检索，最多 2 轮纠错循环。LLM 自检失败时降级为规则检查（答案长度 ≥ 20 字、不含“不知道”等否定表达、检索结果非空）。

3. 你的向量检索和 BM25 检索是怎么融合的？为什么选择 RRF 而不是简单的加权？

回答思路：先讲问题本质，再讲算法选择的理由，最后讲工程细节。

为什么需要融合？

向量检索擅长**语义理解**（“怎么配置权限” ≈ “权限管理操作步骤”），但对精确术语和编号不敏感。

BM25 擅长**精确匹配**（“第 12 条第 3 款”），但对同义词和语义相近表达无能为力。单独使用任何一种，都存在召回盲区。

为什么选 RRF 而不是加权求和？

加权求和（如 $0.6 \times \text{vector_score} + 0.4 \times \text{bm25_score}$ ）有一个根本问题：**两路的分数不可比**。

- 向量检索的 COSINE 相似度在 0~1 之间，分布通常集中在 0.6~0.9
- BM25 的 TF-IDF 分数范围不固定，取决于文档集合的统计特征，可能是 0~50

如果强行加权，BM25 分数高的文档会压倒向量检索结果，或者反过来，需要人工调参才能平衡。而且这个最优权重会随着数据集变化而失效。

RRF 的核心思想是**只看排名位置，不看绝对分数**：

$$\text{score}(d) = \sum 1 / (k + \text{rank}_i(d))$$

排名第 1 的文档得分 $1/(60+1) \approx 0.0164$ ，排名第 10 的得分 $1/(60+10) \approx 0.0143$ 。无论原始分数的尺度如何，融合后的排序都是稳定的。常数 $k=60$ 控制头部和尾部的分数差距， k 越大排名越“平滑”。

工程实现细节

```
// 去重键：内容前 50 字
String deduplicationKey = chunk.getContent().substring(0, Math.min(50, len));

// 如果同一文档同时被 Vector 和 BM25 检索到
```

```

if (existing != null) {
    existing.setSource(Source.HYBRID); // 标记为混合命中
    existing.setScore(existing.getScore() + rrfScore); // 叠加 RRF 分数
}

```

跨路命中（HYBRID）的文档因为在两路都有 RRF 分数贡献，自然排名更高，这正是我们期望的行为——**两种检索路径都认为相关的文档，几乎一定是高质量结果。**

4. AgentToolContext 的 ThreadLocal 设计是怎么回事？解决了什么问题？

回答思路：先讲设计动机，再讲具体机制，最后讲线程安全。

设计动机

我的系统有一个特殊的架构约束：**内部 Agent 和外部 MCP 客户端共享同一组 @ToolBean**。这意味着 DocSearchTool.searchDocs()这个方法可能被两种方式调用：

6. 内部 Agent 调用：LLM 通过 Function Calling 触发，此时需要知道当前查询的 kbIds（知识库范围）和 userId
7. 外部 MCP 调用：Claude Desktop 直接通过 MCP 协议调用，参数由客户端显式传入

问题是：Tool 的方法签名是 LLM 看到的 schema，我不希望让 LLM 操心 kbIds 和 userId 这些业务参数——它只需要决定“用不用这个工具”就够了。但工具内部确实需要这些信息。

解决方案：ThreadLocal 上下文

```

// Agent 执行前激活上下文
AgentToolContext.activate(kbIds, userId);
try {
    // LLM 决定调哪些工具 → Spring AI 自动在当前线程执行
    chatClient.prompt().user(query).call().content();

    // 工具执行时自动把结果写入上下文
    List<RetrievedChunk> chunks = AgentToolContext.get().getChunks();
} finally {
    AgentToolContext.clear(); // 必须清理，防止 ThreadLocal 泄漏
}

```

工具内部的逻辑：

```

@Tool
public List<RetrievedChunk> searchDocs(String query, int topK, List<Long> kbIds) {

```

```

// 优先使用显式参数（MCP 调用），其次使用上下文（Agent 调用）
List<Long> effectiveKbIds = (kbIds != null) ? kbIds :
AgentToolContext.get().getKbIds();

List<RetrievedChunk> results = vectorRetriever.retrieve(query, effectiveKbIds, topK);

// 如果上下文激活，自动收集结果
if (AgentToolContext.isActive()) {
    AgentToolContext.get().addChunks("doc_search", results);
}
return results;
}

```

线程安全

因为 Spring AI 的 Function Calling 可能并发执行多个工具，AgentToolContext 内部的集合使用了 Collections.synchronizedList()和 Collections.synchronizedMap()来保证线程安全。同时 ThreadLocal 本身保证了不同请求之间的隔离。

总结这个设计的好处

8. 工具层解耦：Tool Bean 不需要知道自己是被 Agent 调用还是被 MCP 调用
9. 参数隐式传递：LLM 不需要知道 kbIds 和 userId，降低了 Function Calling 的 Schema 复杂度
10. 结果自动收集：Agent 不需要从 LLM 的返回值中解析工具结果，直接从上下文中获取结构化数据
11. 一套实现，两种接入：同一组 Bean，内部 Agent 和外部 MCP 无缝复用

5. Cross-Encoder 重排序的降级方案是怎么设计的？为什么不直接跳过？

回答思路：先讲为什么不能跳过，再讲降级算法，最后讲效果权衡。

为什么不能直接跳过？

RRF 融合后通常有 20~25 个候选文档，但 LLM 的 Context Window 有 Token 预算限制，我只能取 Top-5~6 个 chunk 进入最终 Prompt。如果跳过重排序，直接按 RRF 分数截取，会有一个严重问题：**RRF 只反映了排名位置的融合，没有对 query-chunk 做细粒度的语义匹配。**

举个例子：一个文档在向量检索中排第 3，在 BM25 中排第 5，RRF 分数很高。但它可能只是"部分相关"——包含了查询中的关键词，但实际内容是另一个话题。Cross-Encoder 通过 (query, chunk)联合编码，能发现这种"表面相关、实际偏题"的情况。

所以重排序不能跳过，必须有一个降级方案。

降级算法设计

当 gte-rerankAPI 不可用时（超时、限流、模型宕机），降级为**关键词频率评分**：

```
// 1. 对 query 按空格/标点分词
List<String> queryTokens = tokenize(query);

// 2. 对每个 chunk 统计关键词命中情况
for (RetrievedChunk chunk : candidates) {
    int hitCount = 0;
    for (String token : queryTokens) {
        if (chunk.getContent().contains(token)) hitCount++;
    }
    float keywordScore = (float) hitCount / queryTokens.size();

    // 3. 混合评分：60% 保留原始分 + 40% 关键词匹配度
    chunk.setRerankScore(0.6f * chunk.getScore() + 0.4f * keywordScore);
}
```

为什么是 60/40 而不是其他比例？

- 60% 原始分：保留 RRF 融合的排序信号，毕竟 RRF 已经综合了多路检索的结果
- 40% 关键词匹配度：引入 query 相关性信号，弥补 RRF 缺少的 query-chunk 直接匹配

这个比例的设计原则是**"降级时保守一点"**——以 RRF 的既有排序为主，关键词匹配只做微调。如果关键词权重太高，遇到同义词表达时会误判。

效果权衡

降级后的排序精度不如 Cross-Encoder（Cross-Encoder 是 SOTA 级别的精排），但比直接用 RRF 截取要好——至少引入了 query-chunk 的直接关联信号。在实际测试中，降级模式的 Top-5 准确率大约是正常模式的 80%，对于一个兜底方案来说是可以接受的。

6. 你的 LLM 模型热切换是怎么实现的？在高并发场景下如何保证正在进行的请求不受影响？

回答思路：先讲实现机制，再讲并发安全，最后讲为什么这样设计。

实现机制

核心是 AiConfigHolder 中的一个 AtomicReference：

```

public class AiConfigHolder {
    private final ConcurrentHashMap<String, String> configMap = new ConcurrentHashMap<>();
    private final AtomicReference<ChatModel> activeModel = new AtomicReference<>();

    public void refreshLlmModel() {
        // 从 configMap 读取最新配置
        String baseUrl = configMap.get("llm.base_url");
        String apiKey = configMap.get("llm.api_key");
        String model = configMap.get("llm.model");
        float temperature = Float.parseFloat(configMap.get("llm.chat_temperature"));

        // 构建新的 ChatModel 实例
        OpenAiApi api = OpenAiApi.builder().baseUrl(baseUrl).apiKey(apiKey).build();
        OpenAiChatModel newModel = OpenAiChatModel.builder()
            .openAiApi(api)
            .defaultOptions(OpenAiChatOptions.builder()
                .model(model).temperature(temperature).build())
            .build();

        // 原子替换
        activeModel.set(newModel);
    }

    public ChatModel getChatModel() {
        return activeModel.get();
    }
}

```

管理员通过后台页面修改模型参数 → 调用 updateBatch()更新 configMap → 调用 refreshLlmModel()构建新模型并原子替换。

高并发下的安全性

这里的关键问题是：**如果一个请求正在用旧模型做流式输出，切换会不会导致它中断？**

答案是**不会**，原因：

12. AtomicReference.set()只是替换引用，不会销毁旧对象。Java GC 只有在没有任何引用指向旧对象时才会回收。
13. 正在执行的请求在调用 getChatModel()时已经拿到了旧模型的引用，并持有这个引用直到 SSE 流结束。AtomicReference.set()替换的是 activeModel 指针，不影响已经持有旧引用的线程。
14. 新请求调用 getChatModel()时会拿到新模型，因为 AtomicReference 保证了可见性 (happens-before) 。

时间线：


```

Thread A: model_ref = getChatModel() → [拿到旧模型] → stream(model_ref) → 完成
          ↑
Admin:      refreshLlmModel() → activeModel.set(新模型)
          ↓
Thread B:      getChatModel() → [拿到新模型] → stream → 完成

```

Thread A 不受影响，Thread B 自动用新模型，切换过程**零停机、零中断**。

为什么不用 Spring 的 @RefreshScope?

@RefreshScope 会销毁并重建 Bean，在高并发下可能导致正在使用该 Bean 的请求抛异常。

AtomicReference 是更轻量、更安全的方案——只替换引用，旧实例自然死亡。

7. Self-Reflection 自检纠错机制的实现细节？在流式输出场景下，答案已经发给用户了怎么办？

回答思路：这是一个很好的"工程妥协"问题，先讲理想方案，再讲流式场景的限制，最后讲实际解法。

理想方案

在非流式场景下，Self-Reflection 的完整流程是：

15. LLM 生成完整答案后，再调一次 LLM，从四个维度评分：

- 事实一致性 (0~10)：答案是否与检索到的来源内容吻合
- 完整性 (0~10)：是否回答了用户的全部问题
- 来源匹配 (0~10)：关键信息是否有据可查
- 表达质量 (0~10)：语言是否清晰、专业

16. 综合置信度 < 0.7 → 判定不通过

17. 不通过时触发重新检索（可能换一组工具组合），最多 2 轮纠错循环

18. 2 轮后仍不通过 → 附加低置信度提示，返回最佳答案

流式场景的现实约束

DocMind 使用 SSE 字符级流式输出，每个 token 生成后立即发送给前端。这意味着**答案已经实时展示给用户了**，无法"撤回重来"。

实际解法

我的处理策略是：

自检仍然执行，但其作用从“触发重新生成”变为以下几点：

19. 追加提示：如果自检不通过，在 SSE 流的末尾追加一条事件，告知前端“本次回答的置信度较低，建议换个问法或检查来源”
20. 日志记录：将自检结果（reflectionLog）持久化到数据库的 qa_message 表中，包含每个维度的评分和具体问题描述。运营团队可以通过管理后台查看低置信度回答的分布，针对性优化知识库
21. 缓存阻断：自检不通过的回答不会被写入热点缓存（RagCacheService），避免低质量答案被反复命中
22. 降级为规则检查：如果 LLM 自检调用本身失败，降级为规则检查——检查答案长度 ≥ 20 字、不含“不知道”/“无法回答”等否定表达、检索结果非空，默认给 0.8 置信度

```
// SelfReflection.java 中的降级逻辑
public ReflectionResult evaluate(String answer, List<RetrievedChunk> chunks, String query)
{
    try {
        // 尝试 LLM 评估
        return llmBasedReflection(answer, chunks, query);
    } catch (Exception e) {
        // 降级为规则检查
        return ruleBasedReflection(answer, chunks);
    }
}

private ReflectionResult ruleBasedReflection(String answer, List<RetrievedChunk> chunks) {
    float confidence = 0.8f;
    List<String> issues = new ArrayList<>();

    if (answer.length() < 20) { confidence -= 0.3f; issues.add("答案过短"); }
    if (answer.contains("不知道") || answer.contains("无法回答")) { confidence -= 0.2f; }
    if (chunks.isEmpty()) { confidence -= 0.2f; issues.add("无检索来源"); }

    return new ReflectionResult(confidence >= 0.7, confidence, issues);
}
```

工程权衡的思考

这其实是**用户体验 vs 答案准确性**的经典权衡：

- 不做流式：可以完美自检纠错，但用户需要等 5~10 秒才看到回答，体验很差

- 做流式但不自检：响应最快，但没有质量保障
- 做流式 + 异步自检：兼顾响应速度和质量审计，通过追加提示、日志分析、缓存阻断来间接提升整体质量

我选择了第三种，因为在企业知识问答场景下，**用户更在意即时反馈**，同时通过持久化自检日志可以驱动离线优化，形成"在线快速响应 + 离线持续改进"的闭环。